

Session 6 : L'Éclairage Statique

💡 Objectifs de la session

Comprendre pourquoi l'éclairage temps réel est trop coûteux pour les grandes scènes, et comment les moteurs modernes précalculent l'éclairage hors ligne (baking). Découvrir les lightmaps, l'Ambient Occlusion, les light probes, et les stratégies d'éclairage mixte. Les exemples pratiques s'appuient sur la documentation de Godot, mais les concepts sont universels — ils s'appliquent à Unity, Unreal Engine, ou tout moteur custom.

Partie 1 : Le Problème — L'Éclairage Temps Réel est Trop Cher

Rappel : le coût de l'éclairage dynamique

En Session 5, nous avons vu que chaque fragment calcule $\vec{N} \cdot \vec{L}$ pour chaque lumière. Pour une scène avec 1000 fragments et 10 lumières, c'est 10 000 produits scalaires — et une scène réelle a des **millions** de fragments. Ajouter les ombres (shadow maps), la spéculaire, et l'atténuation, et le budget de 2 ms explose rapidement.

💡 Le constat des moteurs professionnels

Dans un jeu réel, la majorité de l'éclairage est **statique** : les bâtiments, les murs, le sol ne bougent pas. La lumière du soleil ne change pas (ou rarement). Pourquoi recalculer le même éclairage 60 fois par seconde ? La réponse : on **ne le fait pas**. On le calcule **une fois** hors ligne, on stocke le résultat dans des textures, et on l'affiche gratuitement à l'exécution.

Partie 2 : Les Lightmaps — L'Éclairage Cuit au Four

Principe du Lightmapping

Un **lightmap** est une texture qui stocke l'éclairage précalculé pour chaque surface de la scène. Au lieu de calculer $\vec{N} \cdot \vec{L}$ en temps réel, le GPU se contente de **lire** la couleur dans la texture — une opération $O(1)$, quasiment gratuite.

Le processus se déroule en deux phases :

1. **Offline (baking)** : Un algorithme de ray tracing ou de radiance calcule l'éclairage de chaque surface et l'écrit dans une texture UV.
2. **Runtime (rendering)** : Le fragment shader multiplie la couleur du matériau par la couleur du lightmap. Aucun calcul de lumière.

1. UV2 — Le dépliage de lightmap

Pourquoi un deuxième canal UV ?

Les UV1 servent à appliquer la texture du matériau (diffuse, normal map). Mais une lightmap a besoin d'un **dépliage différent** : chaque surface doit avoir sa propre région dans la texture, sans chevauchement, et avec une densité d'échantillonnage uniforme. C'est le rôle des **UV2** — un second canal UV dédié au lightmap.

- **UV1** : Optimisé pour la répétition de textures (tiling), peut chevaucher.
- **UV2** : Optimisé pour l'éclairage, **aucun chevauchement**, densité uniforme.

💡 Dans Godot

Godot génère automatiquement les UV2 lors du baking si l'option "Generate UV2" est activée sur le MeshInstance3D. On peut aussi fournir des UV2 personnalisés depuis Blender par exemple. Voir la documentation Godot : MeshInstance3D GI Mode.

2. Le shader de lightmap — si simple

Fragment shader avec lightmap (GLSL générique).

```
in vec2 vUV;           // UV1 pour la texture du matériau
in vec2 vUV2;          // UV2 pour le lightmap
uniform sampler2D uDiffuseMap;
uniform sampler2D uLightmap;

out vec4 fragColor;

void main() {
    vec3 albedo = texture(uDiffuseMap, vUV).rgb;
    vec3 bakedLight = texture(uLightmap, vUV2).rgb;
    // Une simple multiplication — c'est tout !
    fragColor = vec4(albedo * bakedLight, 1.0);
}
```

💬 Pourquoi c'est si rapide

Le shader ci-dessus fait **deux lectures de texture et une multiplication**. Aucun produit scalaire, aucune boucle sur les lumières, aucune shadow map. C'est le shader d'éclairage le plus rapide qui existe — et il produit un rendu de qualité **ray tracing** (ombres douces, rebonds de lumière, color bleeding). Le coût : tout est précalculé et immobile.

3. Limitations des lightmaps

⚠️ Ce que les lightmaps ne peuvent pas faire

- **Objets en mouvement** : Un personnage qui marche sur un sol éclairé par lightmap ne reçoit pas l'éclairage du lightmap — il faut un éclairage dynamique séparé.
- **Lumières dynamiques** : Si une lumière s'allume ou s'éteint, le lightmap ne change pas. Il faut rebaker.
- **Mémoire** : Les lightmaps sont des textures, souvent en HDR, qui consomment de la VRAM. Une grande scène peut nécessiter plusieurs lightmaps de 2048×2048.
- **Temps de baking** : Le précalcul peut prendre de quelques secondes à plusieurs heures selon la complexité de la scène et la qualité demandée.

4. Lightmaps dans Godot

Le système LightmapGI

Godot utilise le noeud LightmapGI pour précalculer l'éclairage. Le baking utilise une approche par voxel + ray tracing :

- **Bake Quality** : Low, Medium, High, Ultra — contrôle la densité des voxels et le nombre de rayons.

- **Directional** : Si activé, stocke aussi la direction de l'éclairage (normales de lightmap), ce qui permet aux normal maps d'affecter l'éclairage baked.
- **Interior** : Optimise pour les scènes d'intérieur (pas de lumière du ciel).

Le résultat est stocké dans un fichier `.exr` (HDR) et appliqué automatiquement aux meshes dont le GI Mode est réglé sur Static.

💡 Documentation Godot

- LightmapGI : LightmapGI Class
- Guide d'utilisation : Global Illumination in Godot

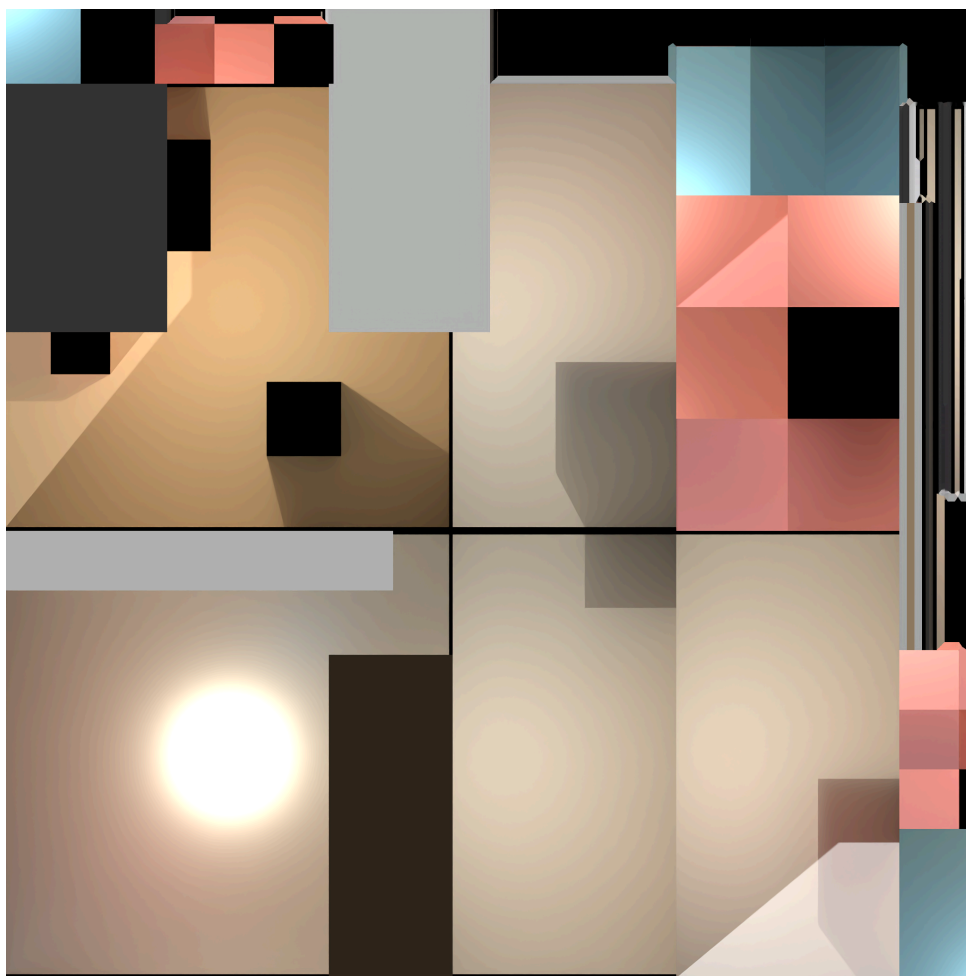


Figure 1: Exemple de lightmap baked dans Blender (Cycles, Combined bake, 2048×2048). Chaque îlot UV correspond à une surface de la scène. On y voit l'éclairage direct (taches claires), les ombres portées, et l'occlusion ambiante dans les coins. Cette texture est multipliée par l'albedo au runtime — aucune lumière temps réel nécessaire.

Partie 3 : L'Ambient Occlusion (AO)

Qu'est-ce que l'Ambient Occlusion ?

L'Ambient Occlusion simule l'assombrissement qui se produit dans les crevasses, les coins et les zones où la géométrie bloque la lumière ambiante. C'est un phénomène **global** : il ne dépend pas d'une lumière particulière, mais de la géométrie environnante.

$$AO(p) = \text{ratio de directions non bloquées vers l'hémisphère au-dessus de } p$$

Plus une surface est entourée d'autres surfaces, plus l'AO est faible (sombre). Les coins de murs, les plis de tissu, les articulations mécaniques sont des zones de fort AO.

5. AO Cuit (Baked AO) vs SSAO

Deux approches pour l'AO

- **Baked AO** : Précalculé hors ligne et stocké dans une texture (comme un lightmap). Qualité parfaite, coût runtime nul. Mais statique — ne s'adapte pas aux objets en mouvement.
- **SSAO (Screen Space Ambient Occlusion)** : Calculé en temps réel dans l'espace écran en utilisant le depth buffer. Approximation, mais dynamique — réagit aux objets en mouvement. Coût GPU non négligeable.

💡 Dans Godot

- **Baked AO** : Intégré automatiquement dans le LightmapGI — l'occlusion ambiante est calculée pendant le baking et stockée dans le lightmap.
- **SSAO** : Activé via Environment > SSAO dans les WorldEnvironment. Voir Environment SSAO.

6. Pourquoi l'AO change tout visuellement

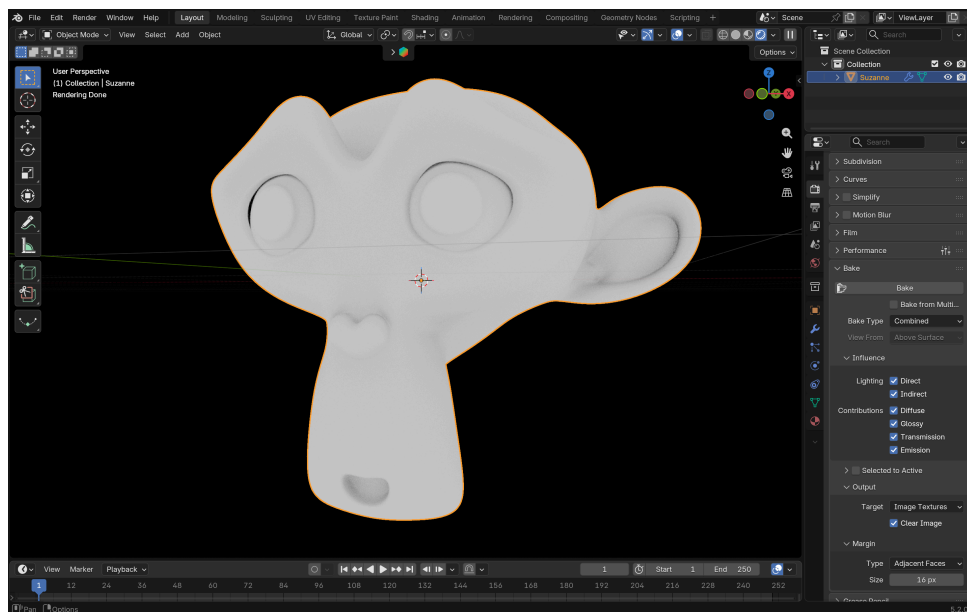


Figure 2: Exemple d'Ambient Occlusion baked dans Blender (Cycles, Bake Type: Ambient Occlusion). Les zones claires reçoivent la lumière ambiante, les zones sombres sont occluses par la géométrie voisine. Remarquez comment les coins, le contact des boîtes avec le sol, et les crevasses sont assombris — c'est ce qui donne le sens du contact et de la profondeur.

L'effet Wadler

Sans AO, une scène 3D paraît “plate” — les coins sont aussi clairs que le centre des murs, les objets semblent flotter. Avec AO, chaque crevasse s’assombrit légèrement, ce qui donne une perception immédiate de **profondeur** et de **contact**. C’est l’un des effets les plus rentables en termes de qualité perçue par rapport au coût de calcul.

Partie 4 : Les Light Probes — L’Éclairage pour les Objets Dynamiques

Le problème des objets en mouvement

Les lightmaps éclairent les surfaces **statiques**. Mais un personnage qui marche dans une scène baked n’a pas de lightmap — il est éclairé par des lumières temps réel, ce qui le détache visuellement de l’environnement. Comment lui donner l’éclairage de la scène sans tout recalculer ?

Light Probes

Les **light probes** sont des points placés dans la scène qui **échantillonnent** l’éclairage baked à leur position. Au runtime, un objet dynamique (personnage, véhicule) interpole l’éclairage entre les probes les plus proches. L’objet reçoit un éclairage qui correspond à son environnement — sans aucun calcul de lumière en temps réel.

- **Placement** : Les probes sont disposées manuellement ou automatiquement dans la scène, surtout autour des zones où des objets dynamiques passent.
- **Stockage** : Chaque probe stocke l’éclairage incoming sous forme d’harmoniques sphériques (SH) — une compression de l’éclairage en 9 coefficients par canal RGB.
- **Interpolation** : Le GPU interpole entre les 4 probes les plus proches (interpolation tétraédrique) pour obtenir l’éclairage à la position de l’objet.

Dans Godot

Godot utilise un noeud `LightmapProbe` placé dans la scène. Les objets dynamiques dont le GI Mode est réglé sur `Dynamic` utilisent automatiquement les probes pour recevoir l’éclairage baked. Voir `LightmapProbe Class`.

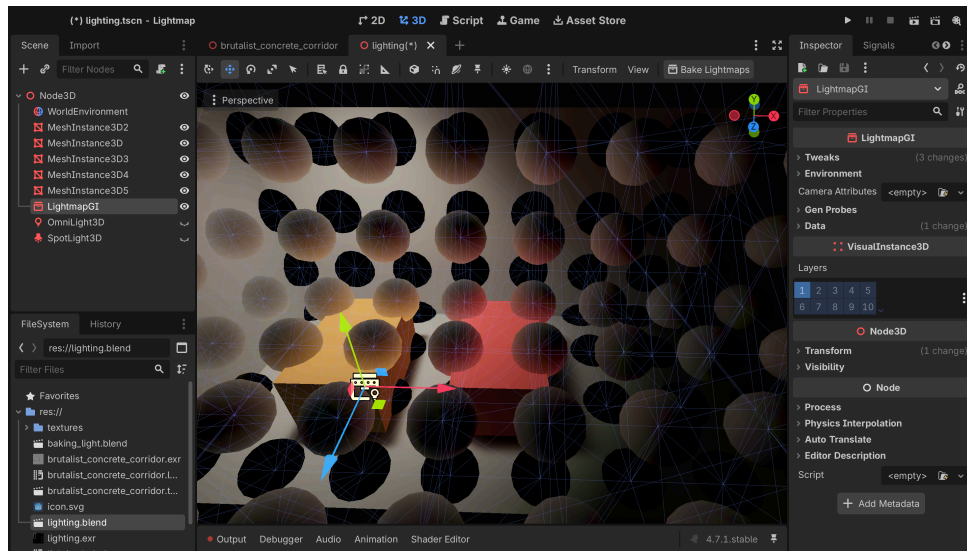


Figure 3: Exemple de placement de light probes dans une scène. Les sphères représentent les probes — chacune capture l'éclairage baked à sa position. Un objet dynamique qui se déplace interpole l'éclairage entre les probes les plus proches, ce qui lui donne un éclairage cohérent avec son environnement sans aucun calcul de lumière temps réel.

7. Spherical Harmonics — La compression magique

Pourquoi pas simplement stocker RGB ?

L'éclairage en un point dépend de la **direction** — une surface pointant vers le haut reçoit la lumière du ciel, une surface pointant vers le sol reçoit la lumière réfléchie du sol. Stocker l'éclairage pour toutes les directions nécessiterait un cube map par probe — beaucoup trop de mémoire.

Les **harmoniques sphériques** (SH) décomposent l'éclairage directionnel en une somme de fonctions de base. Avec seulement 9 coefficients par couleur (27 valeurs au total), on peut reconstruire un éclairage **directionnel** de basse fréquence — suffisant pour l'éclairage diffus d'un personnage. C'est une compression avec un ratio d'environ 100:1 par rapport à un cube map.

Utilisation des SH dans un shader (pseudo-code).

```
// 9 coefficients par canal (27 valeurs au total)
uniform vec3 shCoeffs[9];

// N = normale du fragment
vec3 evalSH(vec3 N) {
    // Basis functions d'ordre 0, 1, 2
    float sh[9];
    sh[0] = 0.282095; // Y_0^0
    sh[1] = 0.488603 * N.y; // Y_1^-1
    sh[2] = 0.488603 * N.z; // Y_1^0
    sh[3] = 0.488603 * N.x; // Y_1^1
    sh[4] = 1.092548 * N.x * N.y; // Y_2^-2
    sh[5] = 1.092548 * N.y * N.z; // Y_2^-1
    sh[6] = 0.315392 * (3.0 * N.z * N.z - 1.0); // Y_2^0
    sh[7] = 1.092548 * N.x * N.z; // Y_2^1
    sh[8] = 0.546274 * (N.x * N.x - N.y * N.y); // Y_2^2

    vec3 color = vec3(0.0);
```

```

for (int i = 0; i < 9; i++)
    color += shCoeffs[i] * sh[i];
return color;
}

```

Partie 5 : L'Éclairage Mixte (Mixed Lighting)

Le meilleur des deux mondes

L'éclairage **mixte** combine l'éclairage baked (statique) et dynamique (temps réel) :

- **Lumières statiques** (soleil, lampes fixes) : Baked dans les lightmaps.
- **Lumières dynamiques** (lampe torche du joueur, explosions) : Calculées en temps réel.
- **Objets statiques** (murs, sol) : Éclairés par lightmap.
- **Objets dynamiques** (personnages) : Éclairés par light probes + lumières temps réel.

C'est l'approche utilisée par la majorité des jeux commerciaux.

8. Les modes d'éclairage mixte dans Godot

GI Mode	Statique	Dynamique	Coût runtime
Disabled	Aucun	Tout temps réel	Élevé
Static	Lightmap + LightmapGI	LightmapProbe pour objets dyn.	Faible
Dynamic	SDFGI + LightmapProbe	SDFGI temps réel	Moyen

Table 1: Les trois modes GI de Godot. Static est l'équivalent du baked — le plus utilisé en production. Dynamic utilise SDFGI (Signed Distance Field GI) pour l'illumination globale temps réel.

🔗 Documentation Godot

- GI Modes : MeshInstance3D GI Mode
- SDFGI : SDFGI in Environment
- Global Illumination : Global Illumination Guide

Partie 6 : Reflection Probes — Les Réflexions Baked

Le problème des réflexions sur objets baked

Un objet avec un matériau métallique ou brillant réfléchit son environnement. En temps réel, les cube maps dynamiques sont trop chers. Solution : précalculer des cube maps à des positions clés de la scène — les **reflection probes**.

Reflection Probes

Une **reflection probe** est un cube map de l'environnement rendu (baked) à une position donnée. Au runtime, un objet brillant utilise la probe la plus proche pour calculer sa réflexion. Comme les light probes, on interpole entre plusieurs probes pour les transitions.

- **Baked** : Précalculée hors ligne. Statique mais gratuite.
- **Realtime** : Mise à jour à l'exécution. Coûteuse mais dynamique.

- **Custom** : Cube map fourni manuellement (ex: HDR environment).

💡 Dans Godot

Godot utilise le noeud `ReflectionProbe` avec la propriété `update_mode` réglée sur `Once` (baked) ou `Always` (temps réel). Voir `ReflectionProbe Class`.

Partie 7 : Le Pipeline Complet d'Éclairage Statique

Workflow type dans un moteur moderne

1. **Préparer la scène** : Placer les lumières statiques, les meshes statiques.
2. **Générer les UV2** : Dépliage automatique ou manuel pour les lightmaps.
3. **Placer les light probes** : Autour des zones de passage des objets dynamiques.
4. **Placer les reflection probes** : Dans les zones avec des matériaux brillants.
5. **Bake** : Lancer le précalcul (secondes à heures selon la scène).
6. **Runtime** : Les objets statiques utilisent les lightmaps, les objets dynamiques utilisent les light probes, les matériaux brillants utilisent les reflection probes.

Partie 8 : Le Texture Baking pour Personnages

Pas seulement les scènes !

Le baking ne sert pas qu'à précalculer l'éclairage des murs et du sol. Il est aussi **indispensable** pour les personnages et objets : on bake des détails d'un mesh **high-poly** vers un mesh **low-poly** pour conserver l'apparence sans le coût géométrique. C'est le workflow standard de tout personnage de jeu vidéo.

9. Le problème : high-poly vs low-poly

Le dilemme du modélisateur

Un personnage sculpté dans ZBrush ou Blender peut avoir 10 millions de polygones — beaucoup trop pour un moteur temps réel. On crée donc une version **low-poly** (5 000 à 50 000 triangles) qui sera utilisée au runtime. Mais la version low-poly perd tous les détails fins : cicatrices, écailles, coutures, rugosité.

Solution : On **bake** les détails de la high-poly vers des textures appliquées sur la low-poly. Le moteur lit ces textures au runtime — les détails réapparaissent sans le coût géométrique.

Étape	Mesh	Résultat
1. Sculpt	High-poly (10M+ tris)	Détails géométriques
2. Retopo	Low-poly (5K-50K tris)	Géométrie légère
3. Bake	High → Low	Textures de détails
4. Runtime	Low-poly + textures	Apparence high-poly

Table 2: Le workflow high-poly → low-poly : on sculpte les détails, on crée une topologie légère, puis on bake les détails dans des textures.

10. Les textures que l'on bake

Les maps essentielles pour un personnage

- **Normal Map** : La plus importante. Encode les détails géométriques (crevasses, bosses, écailles) sous forme de normales perturbées. Le fragment shader les utilise pour simuler du relief sans géométrie supplémentaire. Vue en détail en Session 7.
- **Ambient Occlusion Map** : Assombrit les crevasses du personnage (aisselles, plis, sous les cheveux). Précalculée depuis la géométrie high-poly, elle ajoute du contact visuel même sans éclairage dynamique.
- **Curvature Map** : Détecte les zones convexes (bords, arêtes) et concaves (creux). Utilisée pour l'usure (poussière dans les creux, éclats sur les arêtes) ou le masquage de matériaux.
- **Cavity Map** : Variante d'AO plus locale — ne capte que les crevasses très fines. Souvent mélangée avec l'AO pour un résultat plus riche.
- **Diffuse/Albedo (baked)** : Parfois on bake la couleur depuis le high-poly (ex: polypainting de ZBrush) vers la low-poly. Utile quand le sculpt contient déjà les couleurs.
- **Position Map / ID Map** : Stocke la position ou un identifiant par région du mesh. Sert pour masquer des zones dans le shader (ex: peau vs métal sur un cyborg).

11. Le baking dans Blender

Le moteur Cycles pour le baking

Blender utilise le moteur **Cycles** pour baker des textures. Le processus :

1. **Préparer les UVs** de la low-poly (UV1, non-chevauchants).
2. **Créer une image texture** vide (ex: 2048×2048) dans l'UV Editor.
3. **Sélectionner** d'abord la high-poly, puis la low-poly (ordre important).
4. **Dans le Render Properties** : sélectionner **Cycles**, puis **Bake**.
5. **Bake Type** : choisir le type de map (Normal, Ambient Occlusion, Diffuse, etc.).
6. **Cage** : Blender projette les rayons depuis la low-poly vers la high-poly. Le **cage** définit la distance maximale de projection — trop petit, des détails sont manqués ; trop grand, des artefacts apparaissent.

Bake d'une Normal Map dans Blender (workflow).

1. Importer high-poly et low-poly dans la même scène
2. Sélectionner high-poly → Shift+click low-poly
3. Render Properties > Engine: Cycles
4. Bake > Bake Type: Normal
5. Influence > Space: Tangent (standard pour jeux)
6. Margin: 16px (évite les seams aux bords d'UV)
7. Click "Bake" → l'image se remplit
8. Sauvegarder l'image (PNG ou TGA)

🔍 Le margin (dilatation)

Les UV islands ne sont jamais parfaitement alignés — il y a des espaces entre eux. Sans **margin**, des pixels noirs apparaissent aux coutures. Le margin (ex: 16px) **dilate** la couleur des islands vers l'extérieur pour remplir ces espaces. C'est un réglage critique pour éviter les seams visibles sur le personnage.

12. Utiliser les textures baked dans un moteur

Assemblage dans le matériau

Les textures baked sont combinées dans le matériau du personnage :

- **Normal Map** → branchée sur le Normal du matériau (via un Normal Map node).
- **AO Map** → multipliée avec l'albedo ou ajoutée comme masque d'assombrissement.
- **Curvature Map** → utilisée comme masque pour l'usure, la poussière, ou les effets de bord.

Au runtime, le personnage low-poly + ses textures baked donne l'illusion d'un mesh high-poly — pour une fraction du coût GPU.

💡 Dans Godot

- **Normal Map** : propriété `normal_texture` du `StandardMaterial3D`.
- **AO Map** : propriété `ao_texture` du `StandardMaterial3D`.
- **Roughness/Metallic** : peuvent aussi être baked et branchés sur `roughness_texture` et `metallic_texture`.
- Voir `StandardMaterial3D Documentation`.

Concepts clés à retenir

💡 Les 7 points essentiels

1. **Lightmap** = **éclairage précalculé dans une texture** : Le GPU lit au lieu de calculer. Qualité ray tracing pour le prix d'une texture lookup.
2. **UV2** = **canal UV dédié au lightmap** : Pas de chevauchement, densité uniforme. Les UV1 restent pour les textures de matériau.
3. **Light Probes** = **éclairage pour les objets dynamiques** : Échantillonnent l'éclairage baked à des points clés, interpolé via harmoniques sphériques. Les personnages ne sont plus détachés de la scène.
4. **Ambient Occlusion** = **assombrissement des crevasses** : Baked (parfait, statique) ou SSAO (dynamique, approximatif). L'effet le plus rentable en qualité perçue.
5. **Éclairage mixte** = **baked + temps réel** : Les lumières statiques sont baked, les lumières dynamiques (joueur, explosions) restent temps réel. C'est ce que font les jeux commerciaux.
6. **Reflection probes** = **réflexions baked** : Cube maps précalculés pour les matériaux brillants. Comme les light probes mais pour les réflexions.
7. **Texture baking pour personnages** : On bake les détails du high-poly (normal map, AO, curvature) vers la low-poly. Le personnage léger a l'apparence du sculpt détaillé — c'est le workflow standard de tout jeu commercial.

💡 Question de réflexion pour la prochaine session

“Nous avons vu comment **baker** des normal maps et des AO maps pour les personnages. Mais comment ces textures sont-elles **appliquées** sur une surface ? Comment répéter une texture de brique sur un mur infini ? Comment éviter le flou quand on regarde une texture de loin ? Et quels formats de texture utilise-t-on en production ?”

Réponse : C'est la **texturisation de base**, que nous verrons en Session 7 — UV1, tiling, mipmaps, formats de compression, et sampler filtering.

Travail Pratique : Baked Lighting en Three.js

💡 Objectif du TP

Mettre en pratique les concepts de lightmapping et d'environnement map. Vous allez :

1. Générer une scène dans Blender avec un script Python
2. Baker l'éclairage dans Blender (Cycles)
3. Exporter la scène en .glb
4. Appliquer le lightmap et un environment map en Three.js

Étape 1 : Générer la scène dans Blender

Script Python fourni

Un script `generate_bake_scene.py` est fourni. Il crée :

- Une pièce (sol, 3 murs, plafond)
- 3 boîtes de tailles différentes
- 2 lumières (soleil + ampoule)
- Les UV2 dépliés automatiquement (prêts pour le baking)

Procédure :

1. Ouvrez Blender > onglet **Scripting**
2. Collez le contenu de `generate_bake_scene.py`
3. Exécutez le script (bouton **Run Script**)
4. La scène apparaît avec les objets, lumières et UV2

Étape 2 : Baker le lightmap dans Blender

Baking avec Cycles

1. Dans **Render Properties** : moteur **Cycles**, GPU si disponible
2. Créez une nouvelle image dans l'UV Editor (2048×2048, 32-bit Float si EXR)
3. Sélectionnez tous les meshes (Shift+A > Select All)
4. **Bake** > **Bake Type** : **Combined**
5. **Margin** : 16px
6. Cliquez **Bake** — l'image se remplit
7. Sauvegardez l'image : `session06_lightmap.png` (ou `.exr` pour HDR)

💡 Conseils

- **Combined** bake inclut diffus + ombres + AO en une seule texture
- Si le résultat est trop sombre, augmentez l'énergie des lumières
- Vérifiez que les UV2 sont bien actifs avant de baker

Étape 3 : Exporter en .glb

Export glTF

1. **File > Export > glTF 2.0 (.glb)**
2. Options : cochez **UVs**, décochez **Compression** pour simplifier
3. Nommez le fichier `session06_bake_scene.glb`
4. Placez-le dans le même dossier que les fichiers `Three.js`

Étape 4 : Appliquer le lightmap en Three.js

Le fichier de départ

Un fichier `session06_baked_lighting.js` est fourni avec :

- Le chargement de la scène .glb via `GLTFLoader`
- Une GUI pour comparer lightmap vs lumières temps réel
- Des **TODO ÉTUDIANT** marqués dans le code

Votre mission :

1. Charger la texture de lightmap (`session06_lightmap.png`)
2. L'assigner à `material.lightMap` sur chaque mesh
3. Régler `lightMapIntensity`
4. (Bonus) Charger un environment map HDR avec `RGBELoader`
5. (Bonus) Comparer les FPS : lightmap vs lumières temps réel

⚠ Pièges fréquents

- **Color space** : Le lightmap doit être en `SRGBColorSpace` si bake en sRGB, ou linéaire si EXR
- **UV2** : Three.js utilise le deuxième canal UV automatiquement pour `lightMap` — vérifiez que le .glb contient bien les UV2
- **glTF export** : Assurez-vous que les UV2 sont exportés (option **UVs** cochée)
- **Intensité** : Si la scène est trop sombre/claire, ajustez `lightMapIntensity` (0 à 3)

💡 Résultat attendu

À la fin du TP, vous devriez avoir une scène 3D éclairée **sans aucune lumière temps réel** — tout vient du lightmap. La GUI permet de :

- Activer/désactiver le lightmap
- Activer/désactiver l'environnement map
- Activer/désactiver les lumières temps réel pour comparer

La différence de FPS entre lightmap et lumières temps réel devrait être significative — c'est tout l'intérêt du baking !